OPT2026: 18th Annual Workshop on Optimization for Machine Learning

Guarded Block Transfers for Polytope Distance: Linear Convergence with Explicit Constants and Gap-Certified Screening for SVM Training

Mayank Gupta IAMMAYANKG@GMAIL.COM
Independent Researcher

Abstract

Hard-margin SVM training is the distance problem between two convex hulls, which the Triangle Algorithm (TA) solves with a certified duality gap. We propose *guarded block transfers*: k Mitchell–Demyanov–Malozemov (MDM) transfers per scan are aggregated into one exact line search, guarded by a comparison with the single best transfer at a cost independent of the number of points n, and replaced by an away step when the leading transfer is capacity-clipped. On the Minkowski-difference polytope, where the objective is 1-strongly convex, we prove global linear convergence with explicit constants and no combinatorial swap-step bounds; the per-step contraction is k-independent and only the drop-count bound grows with k, so aggregation is provably safe while the saving in scans is measured, not proved. Since the TA certificate is the Frank–Wolfe gap divided by the current distance, gap-safe screening applies, and we bound the iteration after which it has removed every zero-weight point off the optimal faces by $O\left(\frac{k}{\rho}\log\frac{1}{\tau}\right)$ up to logarithmic factors in the geometry, ρ the rate constant and τ the optimal-face margin. On five LIBSVM benchmarks, with fair baselines (full kernel caches, both LIBLINEAR solvers), the solver matches the baselines to five digits wherever both converge, ties LIBSVM on sparse-support hard margins and beats our BPCG, but our full-cache SMO ties or beats it on kernel L_2 margins, LIBLINEAR's dual solver is up to $4.5\times$ faster on the separable L_2 margin, and it is one to three orders of magnitude slower on dense-support L_2 margins and nearly touching kernel hulls. Its edge is the certificate, not speed.

1. Introduction

Training a linear hard-margin SVM is equivalent to finding the closest points of the two classes' convex hulls: the optimal separator is the perpendicular bisector of the witness pair and the margin is half the hull distance [8]. Kalantari's Triangle Algorithm (TA) exploits this *distance duality* directly [1]: TA I decides whether the hulls intersect, and TA II computes their distance while maintaining an iterate pair (p, q) and a certified interval $[\mathrm{LB}, \mathrm{UB}]$ from parallel supporting hyperplanes. The enhanced TA (ETA) of [2] adds dot-product caching and search heuristics. Below, ETA denotes that solver with our guarded block transfers, K-ETA its kernelised form, and "our SMO" our own SMO implementation.

The workhorse step of such solvers is the *weight transfer* from the worst active vertex to the best candidate, the Mitchell–Demyanov–Malozemov (MDM) step [4], which is the pairwise Frank–Wolfe (FW) step of [3] on the relevant polytope. Each step costs an $O(n)$ scan and yields one transfer; we ask how much of the scan can be reused across several transfers without weakening the guarantee. Our contributions:

© M. Gupta.

1. **Guarded block transfers** (§3): pair the top-k receivers with the worst-k active donors, take one exact line search along the aggregated direction, *guard* it by an $O(k^2)$ comparison with the single MDM step, and fall back to an away step on capacity-clipped iterations.
2. **Linear convergence with explicit constants** (Theorem 3): on $Z = \operatorname{conv}(V) - \operatorname{conv}(W)$ the guarded block algorithm inherits the pyramidal-width rate of pairwise FW up to a constant, and the away fallback replaces the combinatorial swap-step bound by linear drop-step counting (cf. [12]).
3. **Screening activation with explicit constants** (Theorem 6): the TA certificate is the FW gap divided by the distance (Lemma 5), so gap-safe screening [17] applies, and we bound the iteration by which it has removed every zero-weight point off the optimal faces.
4. **A regime study on real data** (§5): two batteries on five LIBSVM datasets locate the boundary from both sides.

Code, proofs, numerical checks and experiment scripts are at `https://github.com/iammayankg/triangle-algorithm-sv`.

2. Setting

Let $V = \{v_1, \dots, v_n\}$, $W = \{w_1, \dots, w_m\} \subset \mathbb{R}^d$. The polytope distance problem is

$$\min_{p \in \operatorname{conv}(V),\ q \in \operatorname{conv}(W)} \tfrac{1}{2}\|p - q\|^2. \tag{1}$$

Writing $z = p - q$ and $Z := \operatorname{conv}(V) - \operatorname{conv}(W) = \operatorname{conv}\{v_i - w_j\}$, (1) is the minimum-norm-point problem $\min_{z \in Z} F(z)$, $F(z) = \frac{1}{2}\|z\|^2$, whose objective is 1-strongly convex and 1-smooth. Let z^* be the minimiser, $\delta^* = \|z^*\|$, $h(z) = F(z) - F(z^*)$, $M = \operatorname{diam}(Z)$ and $\delta = \mathrm{PW}(A) > 0$ the pyramidal width [3] of the atom set $A = \{v_i - w_j\}$ (a property of the atom set, not of Z alone).

The algorithm keeps simplex weights α on V, β on W (iterate z_t after t iterations), with $p = \sum \alpha_i v_i$, $q = \sum \beta_j w_j$; the induced product representation $\lambda_{ij} = \alpha_i \beta_j$ of z over the atoms $v_i - w_j$ has support $\operatorname{supp}\alpha \times \operatorname{supp}\beta$. With $g = -\nabla F(z) = q - p$, the *side scores* are $s_i^V = \langle g, v_i \rangle$ and $s_j^W = \langle p - q, w_j \rangle$, maintained incrementally from cached arrays.

Certificate. For any iterate, $\mathrm{UB} = \|z\|$ and $\mathrm{LB} = \min_{u \in Z}\langle z, u\rangle / \|z\| = \big(\min_i \langle z, v_i\rangle - \max_j \langle z, w_j\rangle\big)/\mathrm{UB}$ satisfy $\mathrm{LB} \le \delta^* \le \mathrm{UB}$; the TA stops when the certified relative gap $(\mathrm{UB} - \mathrm{LB})/\mathrm{UB}$ is at most ε. The L_2 soft margin with regularisation parameter C, the ν-SVM and the kernel variants reduce to (1) exactly (augmented coordinates that turn the Gram matrix K into $K + I/C$; reduced convex hulls; feature-space images), see Appendix B.

Gaps. $g^{\mathrm{FW}} = \max_{u \in Z}\langle g, u\rangle - \langle g, z\rangle$, $g^{\mathrm{A}} = \langle g, z\rangle - \min_{u \in \operatorname{supp}\lambda}\langle g, u\rangle$, $g^{\mathrm{PW}} = g^{\mathrm{FW}} + g^{\mathrm{A}}$. Because $\langle g, v_i - w_j\rangle = s_i^V + s_j^W$ is separable and the support is a product set, each gap splits into a V-side and a W-side term computable from the score arrays in $O(n + m)$ (Lemma 7), written $g_V^{\mathrm{PW}}, g_W^{\mathrm{PW}}$ (likewise for FW and A), with g_{side} the term of the side being stepped; a transfer $v_r - v_u$ on the V side is a pairwise direction of Z (Lemma 8).

Algorithm 1: One iteration of the guarded block-transfer step (side V; W symmetric)

top-k receivers by s^V; worst-k donors in $\operatorname{supp}\alpha$; pair best-with-worst: (r_j, u_j), $j \le k'$ ($k' \le k$ pairs, limited by the support size); pair 1 is the MDM pair (r^*, u^*)

$d_j \leftarrow v_{r_j} - v_{u_j}$; $\mathrm{gap}_j \leftarrow s^V_{r_j} - s^V_{u_j}$; discard pairs with $\mathrm{gap}_j \le 0$ and receivers that are donors (if r^* is a donor it leaves the donor list, so pair 1 survives); $\gamma_j \leftarrow \min(\mathrm{gap}_j/\|d_j\|^2,\ c_j)$

// $c_j = \alpha_{u_j}$: donor capacity

1 **if** *pair 1 uncapped* ($\gamma_1 = \mathrm{gap}_1/\|d_1\|^2$) [case (a)] **then**

2 $\quad$ $\Delta_B \leftarrow$ gain of exact line search $\theta \in [0,1]$ along $d = \sum_j \gamma_j d_j$; $\quad \Delta_1 \leftarrow$ gain of the MDM step

3 $\quad$ take the step with the larger gain // guard, $O(k'^2)$

4 **else**

5 $\quad$ **if** *the away step on u^* is boundary-clipped* [case (b)] **then**

6 $\quad\quad$ take it // drop step: $\alpha_{u^*} \to 0$

7 $\quad$ **else**

8 $\quad\quad$ take the best of {block, MDM, away(u^*), toward(r^*)}

9 $\quad$ **end**

10 **end**

3. Guarded block transfers

Algorithm 1 amortises one $O(n)$ scan over up to k transfers. Only the scan is amortised: a new receiver still costs one Gram or kernel column, $O(nd)$. Every candidate gain is available from cached quantities at a cost independent of n: with $S = \langle g, d\rangle = \sum_j \gamma_j \mathrm{gap}_j$ and $D = \|d\|^2 = \gamma^\top \Gamma \gamma$, Γ the $k' \times k'$ Gram matrix of the pair directions ($O(k'^2)$ cached dot products), the block gain is $\theta S - \theta^2 D/2$ at $\theta = \min(S/D, 1)$; the MDM, away and toward gains are the usual one-dimensional expressions. The *gain* of a step is the decrease $h(z_t) - h(z_{t+1})$. The cache update for a block is one matrix–vector product with the net per-index coefficients; columns are computed on first use and cached, so support columns cost nothing after their first appearance. Feasibility is immediate, since donors lose at most $\theta\gamma_j \le c_j$. We call an iteration *case (a)* if pair 1 is uncapped and *case (b)* otherwise; the away step of line 6, which drives α_{u^*} to zero, is a *drop* step (a capped block pair may also zero a donor, which only helps the counts below).

Safe screening. With $r = \sqrt{\mathrm{UB}^2 - \max(\mathrm{LB}, 0)^2}$ and $v_{\min} = \arg\min_i \langle z, v_i\rangle$, a zero-weight point with $\langle z, v_i\rangle - \langle z, v_{\min}\rangle > r\|v_i - v_{\min}\|$ is outside the optimal support and is removed (symmetrically on W with the max). Because the optimum's support survives, the reduced problem has the same optimum and all subsequent lower bounds remain valid.

4. Analysis

Full proofs and the conventions for degenerate cases are in Appendix A.

Lemma 1 (Block gain) *Let $b_j = \gamma_j\|d_j\|$ over the k' retained pairs, all with $\mathrm{gap}_j > 0$. Then $S \ge \sum_j b_j^2$, $D \le k'S$, and the realised block gain satisfies $\Delta_B \ge S/(2k') \ge \Delta_1/(2k')$, Δ_1 the MDM gain.*

With the guard, every case-(a) iteration gains $\max(\Delta_B, \Delta_1) \geq \Delta_1$: the guarded step never makes less progress than pairwise FW.

Lemma 2 (Good-step progress) *If the iteration works on the side with the larger pairwise gap, then on any non-drop iteration* $\text{gain} \geq \min\big((g^{\text{PW}})^2/(32M^2),\ g^{\text{PW}}/8\big)$.

Theorem 3 (Global linear convergence) *Suppose the guarded block-transfer algorithm with away fallback is run for one side per iteration (the side with the larger pairwise gap), using any $k \geq 1$, from any vertex pair (initial support $s_0 = 2$). Let $h_t = h(z_t)$. Then every non-drop iteration satisfies $h_{t+1} \leq (1-\rho)h_t$ with*

$$\rho = \tfrac{1}{16}\,(\delta/M)^2.$$

For any horizon T, among the first T iterations the number of drop steps is at most $k\,T_{\text{good}} + s_0$, where T_{good} is the number of non-drop iterations; consequently, $h_T \leq h_0\,(1-\rho)^{(T-s_0)/(k+1)}$. Under the two-sided schedule of the implementation (larger-gap side first, second side skipped after a drop) every non-drop iteration has the same contraction, the drop count is at most $2kT_{\text{good}} + s_0$, and $h_T \leq h_0\,(1-\rho)^{(T-s_0)/(2k+1)}$ (Lemma 10).

The rate combines Lemma 2 with $g^{\text{PW}} \geq \delta\sqrt{2h}$ and $g^{\text{PW}} \geq g^{\text{FW}} \geq h$ (Lemma 9). Drop counting is the away-step argument: each drop removes one support index and non-drop steps add at most k. No swap-step bound is needed: a case-(b) iteration either drops (counted above) or takes the best of four candidates, of which the toward or away step alone gains Lemma 2's amount, whichever wins.

Proposition 4 (Near-orthogonal speed-up) *If all k' pairs are uncapped and $|\langle d_i, d_j\rangle| \leq \eta\|d_i\|\|d_j\|$ for $i \neq j$, then the block gain is at least $(\sum_j \Delta_j)/(1+\eta(k'-1))$, where Δ_j is pair j's individual gain.*

As $\eta \to 0$ one scan realises the sum of all k pairwise gains.

Lemma 5 (The certified gap is the FW gap) *At any full scan, with* LB *evaluated at the current $z \neq 0$,* $\text{UB} - \text{LB} = g^{\text{FW}}(z)/\text{UB}$ *(a retained best lower bound gives $\leq$), and $g^{\text{FW}}(z) \leq 2h + R\sqrt{2h}$ with $R = \max_{u\in Z}\|u\|$, the radius of Z about the origin.*

Theorem 6 (Screening activation) *Assume $\delta^* > 0$. Let $F_V = \{i : \langle z^*, v_i\rangle = \min_j \langle z^*, v_j\rangle\}$ and, with the max, F_W be the optimal faces: the vertices of zero score margin, which contain the optimal supports (possibly strictly). Let $\tau_i = \langle z^*, v_i\rangle - \min_j\langle z^*, v_j\rangle$ be the score margin of v_i (and symmetrically on W), with $\tau > 0$ the minimal nonzero margin. Finally, let L be the larger class diameter.*

A zero-weight point with margin τ_i is screened as soon as $r < \tau_i/(3L)$. Consequently, once $h_t < H := \tau^4\delta^{*2}/(2592\,R^4L^4)$, every zero-weight point of V outside F_V and of W outside F_W is removed permanently at the next screening round. Under Theorem 3 this holds for every iteration $t > s_0 + \frac{a_k}{\rho}\log\max\{1, h_0/H\}$, with $a_k = k+1$ one-sided and $2k+1$ two-sided. Following this iteration, each subsequent scan costs $O(|F_V| + |F_W| + |\operatorname{supp}\alpha| + |\operatorname{supp}\beta|)$ instead of $O(n+m)$.

In words: since $r^2 \leq 2R(\text{UB} - \text{LB})$, a point with margin τ_i is screened once the certified gap falls below $\tau_i^2/(18RL^2)$. The faces can strictly contain the supports: for $V = \{(1,0),(1,1),(2,0)\}$ and $W = \{0\}$, $z^* = (1,0)$ and $(1,1)$ has zero weight and zero margin, so no gap-safe rule can remove it. Lemmas 1, 5 and 10, Proposition 4 and Theorem 6 are verified numerically on random and adversarial ensembles at machine precision (Appendix E).

Table 1: Regime battery on real data, five seeds, 600 s per solver, mean $\pm$ 95% CI. Kernel cells (RBF with width parameter $1/d$) use a class-balanced subsample of n_k points. Same objective per row: LIN, linear hard margin (ETA with screening vs LIBSVM's $C{=}10^6$ surrogate; our SMO in brackets); KHM, feature-space hard margin (K-ETA vs the same surrogate); KL2, kernel L_2 margin, $C{=}1$ (K-ETA vs kernel SMO, exact objective; in brackets LIBSVM on the precomputed $K + I/C$ kernel, fit time + kernel construction, September machine). conv = seeds in which ETA certified; oracle = $O(nd)$ column (ETA) or row (SMO) evaluations, thousands; both cache every column or row they compute. Released code, AMD EPYC 9754 (Appendix C).

cell	dataset	n_k / d	ETA (s)	baseline (s)	conv	dist. ETA / base	acc. ETA / base	oracle
LIN	gisette	6000 / 5000	38.2 ± 3.2	38.5 ± 3.1 [34.0 ± 3.2]	5/5	3.6638 / 3.6640	.981 / .981	1.5 [1.4]
KHM	gisette	6000 / 5000	64.6 ± 6.2	75.6 ± 3.1	5/5	.05215 / .05215	.979 / .979	2.6
KHM	ijcnn1	9853 / 22	242 ± 69	1.1 ± 0.2	5/5	.00430 / .00430	.946 / .946	2.6
KHM	a9a	10000 / 123	> 600	18.5 ± 2.0	0/5	$\leq$.00002 / .0033	.685 / .752	6.0
KHM	covtype	10000 / 54	> 600	122 ± 33 (maxiter)	0/5	$\leq$.00044 / .00025	.711 / .712	7.6
KHM	w8a	6479 / 300	7.2 ± 20.1	—	5/5	hulls intersect	—	0.5
KL2	gisette	6000 / 5000	89.8 ± 7.8	90.6 ± 8.5 [$0.5 \pm 0.1 + 3$]	5/5	.06882 / .06882	.971 / .971	3.7 / 3.6
KL2	w8a	6479 / 300	6.3 ± 1.0	5.2 ± 1.5 [$0.6 \pm 0.1 + 1$]	5/5	.05682 / .05682	.9745 / .9745	3.6 / 3.6
KL2	a9a	10000 / 123	13.2 ± 1.0	8.9 ± 1.5 [$2.2 \pm 0.2 + 2$]	5/5	.03019 / .03019	.795 / .795	7.6 / 7.6
KL2	covtype	10000 / 54	16.8 ± 4.3	5.8 ± 1.2 [$2.1 \pm 0.1 + 2$]	5/5	.02609 / .02609	.792 / .792	9.1 / 9.1
KL2	ijcnn1	9853 / 22	5.3 ± 0.3	2.6 ± 0.3 [$1.1 \pm 0.1 + 2$]	5/5	.04790 / .04790	.959 / .959	4.6 / 4.5

5. Experiments

Setup. All solvers are validated against an exact QP (distances agree to 10^{-9} or better). Stopping rule $\varepsilon = 10^{-3}$ on the certified relative gap unless stated; baselines use matched tolerances on the same objective, LIBLINEAR up to its intercept penalty (Appendix C). Synthetic instances follow [2] (two Gaussian clouds, $n = 5{,}000$ per class). Real data are the LIBSVM sets a9a, w8a, ijcnn1, gisette and covtype (100k subsample), standardised, official test sets where they exist, five seeds, 600 s per solver and cell, on two AMD EPYC machines (Appendix C says which ran what).

Blocks and screening (Appendix C). In isolated runs (Table 6) the guard and fallback alone (guarded $k{=}1$) cut plain pairwise FW's iterations 1.4–2.7$\times$ but its time by at most 1.3$\times$; aggregation cuts iterations a further 3.8–7.0$\times$, which buys wall-clock only where per-iteration $O(n)$ work, not column computation, dominates: on the four kernel cells 2.1–2.8$\times$ over guarded $k{=}1$ and 1.2–1.9$\times$ over exact-search MDM, nothing on gisette, where columns take 82–89% of the time. Blocks cut iterations 6–19$\times$ over MDM on real data and 7–19$\times$ on synthetic; the cut is smallest where drops are frequent (36% of gisette's iterations) or pair directions correlate (measured 0.26–0.29 on ijcnn1 and covtype, Proposition 4); a cost-model choice of k is in Appendix C. Screening gives 1.3–2.4$\times$ at $\varepsilon = 10^{-5}$ and is neutral at 10^{-3}.

Real data, sparse-support regime (Table 1). TA I first certifies separability (gisette yes, the other four no, in 0.1–30 s). On gisette ($d{=}5{,}000$; 1,237 of 6,000 points in the support) ETA ties LIBSVM on the linear hard margin and our full-cache SMO is 1.1$\times$ faster than both (overlapping intervals); K-ETA is 1.2$\times$ faster than LIBSVM on the RBF hard margin, with identical solutions. On the kernel L_2 margins K-ETA computes as many kernel columns as kernel SMO computes rows; SMO

ties it on gisette and w8a and is 1.5–$2.9\times$ faster elsewhere. (Our submission reported 4–$6\times$ fewer columns and wins of up to $3\times$: that was SMO's 512-row cache.) Against BPCG [12] with the same column cache and certificate (Table 7), block ETA needs 11–$35\times$ fewer iterations and is 2.5–$7.1\times$ faster on the kernel cells, a tie on the gisette L_2 margin. LIBSVM on the precomputed $K + I/C$ kernel (brackets) was 2.7–$22\times$ faster than K-ETA on the September machine.

The KHM column shows where the geometry turns against the method. On ijcnn1 both return the same solution (819 support vectors), but K-ETA needs 1.5–2.7×10^5 iterations and 242 s against LIBSVM's 1.1 s: the hulls are 0.004 apart in a space of diameter $\approx \sqrt{2}$, so a relative certificate needs an absolute gap of order 10^{-6}, which suits LIBSVM's second-order selection far better. On a9a and covtype the hulls are within 10^{-4} and 5×10^{-4} of touching, and neither solver certifies anything. On w8a the hulls intersect (duplicate points, opposite labels); K-ETA certifies this in under 0.1 s in four seeds and 36 s in the fifth.

Real data, L_2 soft margins (Table 2, appendix). The same boundary appears within one problem class. Gisette is separable, so its L_2 margin has a sparse support and ETA converges to the certified gap in 19–21 s at every C; our full-cache SMO is 1.1–$1.3\times$ faster. LIBLINEAR's primal Newton solver, which the battery ran, needs 4 min to 1.2 h at the matched tolerance 10^{-6} and stops 3–24% above the optimum at its default 10^{-4}; its dual solver reaches the certified optimum in 3–4.5 s at every tolerance, 2.9–$4.5\times$ faster than ETA on the same machine (Tables 4 and 6). On gisette ETA adds only the certificate. On the four overlapping datasets LIBLINEAR finishes in 0.1–24 s; ETA converges within the cap only on a9a (four of five seeds) and ijcnn1 at $C{=}0.1$ (583 and 336 s, against our SMO's 163 and 59 s), at $C{=}10$ its capped iterate loses 1–27 accuracy points, and on covtype it is not usable at any C. Uncapped, ETA certifies 10^{-3} on a9a, w8a and ijcnn1 at $C{=}0.1$ in 259–792 s, 30–$1{,}800\times$ LIBLINEAR's time (Figure 1, Table 5), at a rate at least linear but with $O(n^2)$-type cost on these dense supports (heuristic in the caption of Figure 1).

6. Related work

Distance duality and the TA [1, 2]; MDM, Gilbert and Wolfe [4, 5, 25, 26]; FW for polytope distance and SVMs [6, 8, 21–24]; away steps, pyramidal width and facial distance [3, 7, 27]. Boosted FW [13], which also aggregates atoms into one direction, swap-step-free pairwise variants [12, 14] and block-coordinate FW [9] are the closest algorithmic relatives; gap-safe screening [15, 17, 18] and active-set identification [19, 20] the closest to Theorem 6, with [10, 11, 16] as its SVM ancestors (Appendix D).

7. Conclusion

Guarded block transfers give the Triangle Algorithm a linear rate with explicit constants and no combinatorial swap-step bounds, and a safe screening rule with a finite activation bound. Rerun with fair baselines, the experiments locate the boundary and narrow the niche: on sparse supports and well-separated hulls the solver ties LIBSVM and beats our BPCG, but a full-cache SMO and LIBLINEAR's dual solver are as fast or faster; on dense supports or nearly touching hulls LIBLINEAR or LIBSVM is one to three orders of magnitude faster. What it adds is the certificate and the intersection test, not speed. Limitations: screening pays only when the certificate closes; the code is NumPy-level and dense; the constants are loose; the ν-SVM variant is unanalysed; k is fixed (Appendix C gives a cost-model choice).

References

[1] B. Kalantari. A characterization theorem and an algorithm for a convex hull problem. *Annals of Operations Research*, 226(1):301–349, 2015.

[2] M. Gupta and B. Kalantari. An enhanced triangle algorithm for large-scale support vector machine optimization. *Journal of Intelligent Decision Making and Information Science*, 3(9s), 2026.

[3] S. Lacoste-Julien and M. Jaggi. On the global linear convergence of Frank–Wolfe optimization variants. *NeurIPS*, 2015.

[4] B. F. Mitchell, V. F. Demyanov, and V. N. Malozemov. Finding the point of a polyhedron closest to the origin. *SIAM J. Control*, 12(1):19–26, 1974.

[5] E. G. Gilbert. An iterative procedure for computing the minimum of a quadratic form on a convex set. *SIAM J. Control*, 4(1):61–80, 1966.

[6] S. S. Keerthi, S. K. Shevade, C. Bhattacharyya, and K. R. K. Murthy. A fast iterative nearest point algorithm for support vector machine classifier design. *IEEE Trans. Neural Networks*, 11(1):124–136, 2000.

[7] J. Guélat and P. Marcotte. Some comments on Wolfe's 'away step'. *Mathematical Programming*, 35(1):110–119, 1986.

[8] K. P. Bennett and E. J. Bredensteiner. Duality and geometry in SVM classifiers. *ICML*, 2000.

[9] S. Lacoste-Julien, M. Jaggi, M. Schmidt, and P. Pletscher. Block-coordinate Frank–Wolfe optimization for structural SVMs. *ICML*, 2013.

[10] L. El Ghaoui, V. Viallon, and T. Rabbani. Safe feature elimination in sparse supervised learning. *Pacific J. Optimization*, 8(4):667–698, 2012.

[11] K. Ogawa, Y. Suzuki, and I. Takeuchi. Safe screening of non-support vectors in pathwise SVM computation. *ICML*, 2013.

[12] K. Tsuji, K. Tanaka, and S. Pokutta. Pairwise conditional gradients without swap steps and sparser kernel herding. *ICML*, 2022.

[13] C. W. Combettes and S. Pokutta. Boosting Frank–Wolfe by chasing gradients. *ICML*, 2020.

[14] F. Rinaldi and D. Zeffiro. Avoiding bad steps in Frank–Wolfe variants. *Computational Optimization and Applications*, 84(1):225–264, 2023.

[15] O. Fercoq, A. Gramfort, and J. Salmon. Mind the duality gap: safer rules for the Lasso. *ICML*, 2015.

[16] J. Wang, P. Wonka, and J. Ye. Scaling SVM and least absolute deviations via exact data reduction. *ICML*, 2014.

[17] E. Ndiaye, O. Fercoq, A. Gramfort, and J. Salmon. Gap safe screening rules for sparsity enforcing penalties. *JMLR*, 18(128):1–33, 2017.

[18] A. Shibagaki, M. Karasuyama, K. Hatano, and I. Takeuchi. Simultaneous safe screening of features and samples in doubly sparse modeling. *ICML*, 2016.

[19] I. M. Bomze, F. Rinaldi, and D. Zeffiro. Active set complexity of the away-step Frank–Wolfe algorithm. *SIAM J. Optim.*, 30(3):2470–2500, 2020.

[20] D. Garber. Revisiting Frank–Wolfe for polytopes: strict complementarity and sparsity. *NeurIPS*, 2020.

[21] B. Gärtner and M. Jaggi. Coresets for polytope distance. *SoCG*, 2009.

[22] K. L. Clarkson. Coresets, sparse greedy approximation, and the Frank–Wolfe algorithm. *ACM Trans. Algorithms*, 6(4):63, 2010.

[23] I. W. Tsang, J. T. Kwok, and P.-M. Cheung. Core vector machines: fast SVM training on very large data sets. *JMLR*, 6:363–392, 2005.

[24] R. Ñanculef, E. Frandi, C. Sartori, and H. Allende. A novel Frank–Wolfe algorithm. Analysis and applications to large-scale SVM training. *Information Sciences*, 285:66–99, 2014.

[25] J. López and J. R. Dorronsoro. Linear convergence rate for the MDM algorithm for the nearest point problem. *Pattern Recognition*, 48(4):1510–1522, 2015.

[26] P. Wolfe. Finding the nearest point in a polytope. *Mathematical Programming*, 11(1):128–149, 1976.

[27] J. Peña and D. Rodríguez. Polytope conditioning and linear convergence of the Frank–Wolfe algorithm. *Mathematics of Operations Research*, 44(1):1–18, 2019.

Acknowledgements. We thank the OPT 2026 reviewers; one of them supplied the example after Theorem 6.

AI assistance. AI tools were used in the preparation of the code, experiments and manuscript; the author verified all results and is responsible for the content.

Appendix A. Proofs

Conventions. $M > 0$ (a singleton Z is solved at initialisation), and the algorithm stops with "intersect" when $\mathrm{UB} = \|z\| = 0$, before any division by UB. Lemma 1 and Algorithm 1 presuppose $k' \geq 1$: a side with no positive-gap pair is skipped, and on a non-drop iteration the side with the larger pairwise gap has one. An away direction whose donor has weight one is unavailable (the iterate is that vertex and the away gap of that side is zero). In Proposition 4, $\eta \geq 0$.

Lemma 7 (Separability) *With the product representation,* $g^{\mathrm{PW}} = g_V^{\mathrm{PW}} + g_W^{\mathrm{PW}}$ *where* $g_V^{\mathrm{PW}} = \max_i s_i^V - \min_{i \in \operatorname{supp} \alpha} s_i^V \geq 0$ *and* g_W^{PW} *likewise; the same holds for* g^{FW} *and* g^{A}.

Proof. $\langle g, v_i - w_j \rangle = s_i^V + s_j^W$ is separable and the support is a product set, so the max over atoms and the min over the support separate. Each side term is nonnegative: the FW term is the maximum score minus the weighted mean of the support scores, the away term is that mean minus the support minimum, and a weighted mean lies between the minimum and the maximum. ∎

Lemma 8 (Side transfers are Z-pairwise directions) *For $\alpha_u > 0$ and any $b \in \operatorname{supp}\beta$, $v_r - v_u = (v_r - w_b) - (v_u - w_b)$ is a difference of an atom of Z and an atom in the support, and the transfer $\gamma \le \alpha_u$ realises $z \leftarrow z + \gamma(v_r - v_u)$ inside Z. The side capacity $\alpha_u = \sum_b \alpha_u \beta_b$ is the total weight of the product atoms $v_u - w_b$, $b \in \operatorname{supp}\beta$: the side transfer is the sum of the atom-pair transfers of masses $\gamma\beta_b$ from $v_u - w_b$ to $v_r - w_b$, and for $r \neq u$ this weight update is feasible exactly when $0 \le \gamma \le \alpha_u$; hence every such transfer remains in Z.*

Proof of Lemma 1. Since $\gamma_j \le \operatorname{gap}_j/\|d_j\|^2$, we have $\gamma_j \operatorname{gap}_j \ge \gamma_j^2\|d_j\|^2 = b_j^2$. Consequently, $S = \sum_j \gamma_j \operatorname{gap}_j \ge \sum_j b_j^2$. By the triangle inequality, $\|d\| \le \sum_j \gamma_j \|d_j\| = \sum_j b_j$, and by Cauchy–Schwarz, $(\sum_j b_j)^2 \le k' \sum_j b_j^2 \le k'S$; hence $D \le k'S$. Exact line search on the 1-smooth F gives $\Delta_B = S^2/(2D) \ge S/(2k')$ in the interior case and $\Delta_B = S - D/2 \ge S/2$ at the boundary $\theta = 1$, where $D \le S$. Finally, $S \ge \gamma_1 \operatorname{gap}_1 \ge \Delta_1$, since all terms are nonnegative and $\Delta_1 = \gamma_1 \operatorname{gap}_1 - \gamma_1^2\|d_1\|^2/2$. ∎

Proof of Lemma 2. Let $g_S = \max(g_V^{\mathrm{PW}}, g_W^{\mathrm{PW}}) \ge g^{\mathrm{PW}}/2$. Every direction below is a difference of two points of one class hull, so its norm is at most $\operatorname{diam}(\operatorname{conv} V) \le M$. For a feasible direction with directional derivative $\sigma > 0$, squared norm D and maximal step $\bar\theta$, exact line search on a 1-smooth function gives $\text{gain} \ge \min(\sigma^2/(2D),\, \bar\theta\sigma/2)$ (interior optimum, or the clipped step evaluated with $\bar\theta D \le \sigma$). In case (a), the guard's gain is at least the MDM gain, whose derivative is g_S and whose interior optimum is attained, yielding $\text{gain} \ge g_S^2/(2M^2) \ge (g^{\mathrm{PW}})^2/(8M^2)$. In case (b), non-drop, $g_S = g_{\text{side}}^{\mathrm{FW}} + g_{\text{side}}^{\mathrm{A}}$, so one term is at least $g_S/2 \ge g^{\mathrm{PW}}/4$. If it is the FW term, the toward step (maximal step 1) gives $\text{gain} \ge \min((g_{\text{side}}^{\mathrm{FW}})^2/(2M^2), g_{\text{side}}^{\mathrm{FW}}/2) \ge \min((g^{\mathrm{PW}})^2/(32M^2), g^{\mathrm{PW}}/8)$; if it is the away term, the away line search is interior (non-drop) and $\text{gain} \ge (g_{\text{side}}^{\mathrm{A}})^2/(2M^2) \ge (g^{\mathrm{PW}})^2/(32M^2)$. The algorithm takes the best available candidate. ∎

Lemma 9 (Geometric strong convexity; [3, Thm. 6] with $\mu = 1$) *For every $z \in Z$ with a representation whose support contains an away atom, $g^{\mathrm{PW}}(z) \ge \delta\sqrt{2h(z)}$ and $g^{\mathrm{PW}}(z) \ge g^{\mathrm{FW}}(z) \ge h(z)$.*

Proof. This is Theorem 6 of [3] for a 1-strongly convex objective; we include the short argument. If $z = z^*$ both inequalities are immediate ($h = 0$ and all gaps are nonnegative). Otherwise let $e = (z^* - z)/\|z^* - z\|$; the pyramidal-width inequality in the proof of [3, Thm. 6] (Definition 3 there, applied to the direction g together with $\langle g, e\rangle \le \|g\|$) gives, for the FW atom s and the away atom v, $\langle g, s - v\rangle \ge \delta\langle g, e\rangle$. By 1-strong convexity, $\langle g, z^* - z\rangle \ge h + \frac{1}{2}\|z^* - z\|^2$, so with $\varrho = \|z^* - z\|$, $g^{\mathrm{PW}} \ge \delta(h/\varrho + \varrho/2) \ge \delta\sqrt{2h}$ by AM–GM. Finally $g^{\mathrm{PW}} = g^{\mathrm{FW}} + g^{\mathrm{A}} \ge g^{\mathrm{FW}} = \max_u \langle g, u - z\rangle \ge \langle g, z^* - z\rangle \ge h$ by convexity. ∎

Proof of Theorem 3. By Lemmas 2 and 9 a non-drop iteration gains at least

$$\min\left(2\delta^2 h/(32M^2),\, h/8\right) = (\delta/M)^2 h/16$$

using $\delta \le M$; hence $h_{t+1} \le (1 - \rho)h_t$. Drop counting: non-drop iterations add at most k support indices, drop iterations add none and remove at least one, and the support size stays ≥ 2 from s_0; so the number of removals, hence of drops, is at most $kT_{\mathrm{good}} + s_0$, so that

$$T_{\mathrm{good}} \ge (T - s_0)/(k + 1).$$

All steps, drops included, are descent steps, so h is monotone. ■

Lemma 10 (Two-sided schedule) *Let every iteration compute both side gaps, take the guarded step of Algorithm 1 on the side with the larger pairwise gap first, and take the other side's guarded step only if the first step was not a drop; an iteration counts as a drop iff its first step is a drop. Then every non-drop iteration gains at least* $\min((g^{\mathrm{PW}})^2/(32M^2),\, g^{\mathrm{PW}}/8)$ *and the number of drops is at most* $2kT_{\mathrm{good}} + s_0$*; hence Theorem 3 holds for this schedule with the same* ρ *and exponent* $(T - s_0)/(2k + 1)$.

Proof. On a non-drop iteration the first step is exactly the step analysed in Lemma 2, so it gains at least the stated amount; the second step is an exact line search along a feasible descent direction (or is skipped), so it cannot increase h. Drop counting: a drop iteration takes a single away step that removes one support index and adds none, because the second side is skipped; a non-drop iteration takes two guarded steps and adds at most k indices each. Removals, hence drops, therefore number at most $2kT_{\mathrm{good}} + s_0$. ■

Remark 11 *The skip is what makes the count work. If the second side always steps, an iteration whose analysed side drops can add up to* k *indices on the other side, and the count becomes* $T_{\mathrm{drop}} \le 2kT_{\mathrm{good}} + kT_{\mathrm{drop}} + s_0$*, which is vacuous for* $k \ge 1$.

Proof of Proposition 4. $D = \sum_{i,j} \gamma_i\gamma_j\langle d_i, d_j\rangle \le \sum_j b_j^2 + \eta\sum_{i\neq j} b_ib_j \le (1+\eta(k'-1))\sum_j b_j^2 \le (1+\eta(k'-1))S$ by Cauchy–Schwarz on the cross terms. If $D \ge S$ the line search is interior and $\Delta_B = S^2/(2D) \ge S/(2(1+\eta(k'-1)))$; if $D < S$ then $\theta = 1$ and $\Delta_B = S - D/2 > S/2$, which is larger. For uncapped pairs $S = \sum_j 2\Delta_j$. ■

Proof of Lemma 5. $\mathrm{LB} = \min_{u\in Z}\langle z, u\rangle/\|z\|$ by definition of the supporting hyperplane bound and $\mathrm{UB} = \|z\|$; subtracting, $\mathrm{UB} - \mathrm{LB} = (\langle z, z\rangle - \min_u\langle z, u\rangle)/\|z\| = g^{\mathrm{FW}}/\mathrm{UB}$. At the optimum $0 = \langle z^*, z^*\rangle - \min_u\langle z^*, u\rangle$; subtracting from $g^{\mathrm{FW}}(z)$, $g^{\mathrm{FW}}(z) = (\|z\|^2 - \|z^*\|^2) + (\min_u\langle z^*, u\rangle - \min_u\langle z, u\rangle) \le 2h + \max_u\langle z^* - z, u\rangle \le 2h + R\|z - z^*\| \le 2h + R\sqrt{2h}$ by 1-strong convexity. ■

Proof of Theorem 6. Strong convexity gives $\|z - z^*\| \le r$ for the screening radius $r = \sqrt{\mathrm{UB}^2 - \max(\mathrm{LB}, 0)^2}$ (τ is defined only if some point has positive margin; otherwise there is nothing to screen). Let $m^* = \min_j\langle z^*, v_j\rangle$, choose $j' \in F_V$ and set $v' = v_{j'}$; since $v_{\min}$ minimises $\langle z, \cdot\rangle$, $\langle z^*, v_{\min}\rangle - m^* = \langle z, v_{\min} - v'\rangle + \langle z^* - z, v_{\min} - v'\rangle \le rL$. Hence $\langle z, v_i - v_{\min}\rangle = \langle z^*, v_i - v_{\min}\rangle + \langle z - z^*, v_i - v_{\min}\rangle \ge \tau_i - rL - r\|v_i - v_{\min}\|$, and the rule fires when this exceeds $r\|v_i - v_{\min}\|$, for which $\tau_i > 3rL$ suffices since $\|v_i - v_{\min}\| \le L$. For the activation time, $r^2 \le 2R(\mathrm{UB} - \mathrm{LB})$ (for $\mathrm{LB} \ge 0$, $r^2 = (\mathrm{UB} - \mathrm{LB})(\mathrm{UB} + \mathrm{LB})$ and $\mathrm{UB} + \mathrm{LB} \le 2R$; for $\mathrm{LB} < 0$, $r^2 = \mathrm{UB}^2$ and $\mathrm{UB} \le 2(\mathrm{UB} - \mathrm{LB})$), and $2R(\mathrm{UB} - \mathrm{LB}) \le 2Rg^{\mathrm{FW}}/\mathrm{UB} \le (2R/\delta^*)(2h + R\sqrt{2h}) \le (4R^2/\delta^*)\sqrt{2h}$ (Lemma 5, with equality in the first step for the freshly evaluated bound; the last step uses $h \le R^2/2$, which holds for every feasible iterate since $h \le \frac{1}{2}\|z\|^2$); requiring $r^2 \le \tau^2/(9L^2)$ yields $h_t < H$, and Theorem 3 with $\log(1/(1-\rho)) \ge \rho$ converts it into the iteration bound (the $\max\{1, \cdot\}$ covers $h_0 \le H$, where the condition holds from the start by monotonicity). The removal is executed at the first screening round after the condition holds; permanence follows from the safety of the rule. ■

Appendix B. Exact reductions

L_2 soft margin. The problem $\min \frac{1}{2}\|w\|^2 + \frac{C}{2}\sum_i \xi_i^2$ is the hard-margin problem on the augmented points $\tilde{x}_i = [x_i;\, e_i/\sqrt{C}]$, e_i the i-th standard basis vector, whose Gram matrix is $K + I/C$ (K the Gram matrix of the original points) and whose augmented hulls never intersect; let δ_C denote their distance. The augmentation is never materialised: the extra coordinates of an iterate are exactly its convex weights, so every augmented inner product is a cached original-space product plus a sparse diagonal correction, $\langle \tilde{p}, \tilde{v}_i \rangle = \langle p, v_i \rangle + \alpha_i/C$ and $\|\tilde{p}\|^2 = \|p\|^2 + \|\alpha\|^2/C$. The separator is recovered as $w = 2(p-q)/\delta_C^2$, $b = (\|\tilde{q}\|^2 - \|\tilde{p}\|^2)/\delta_C^2$, and the primal value is $P = 2/\delta_C^2$. Validation: the solver agrees with an exact QP on the explicitly augmented problem to 10^{-10} across random instances and values of C; the regularisation path δ_C is monotone in C; on a9a (C=0.1) its certified primal 687.07 equals LIBLINEAR's to five digits.

ν-SVM. The L_1 (hinge) soft margin is the nearest-point problem between *reduced* convex hulls $R(V, \mu) = \{\sum a_i v_i : \sum a_i = 1,\ 0 \le a_i \le \mu\}$, $\mu = 2/(\nu\ell)$ with $\ell = n + m$ the number of training points [8]. Two changes: the extreme-point oracle becomes a capped top-k blend (weight μ on the $\lfloor 1/\mu \rfloor$ best points in the search direction, via a partial sort, still $O(n)$), and the primary step is the capped MDM transfer with blended toward steps as fallback; caching and the reduced support-function lower bound carry over unchanged. Validation: agreement with the exact box-constrained QP to 10^{-9}–10^{-14} and with LIBSVM's NuSVC to $\sim 10^{-10}$ through the mapping $\delta^* = \|w_{\mathrm{svc}}\|/\varsigma$, w_{svc} LIBSVM's weight vector and ς the per-class sum of the rescaled dual coefficients. The reduced hull is a polytope, so (1) still applies, but its vertices are blends and the capped transfer is not the pairwise step analysed in §4; the analysis of this variant is left open.

Kernels. The algorithm touches data only through inner products, so kernelisation changes plumbing, not algorithm: Gram columns become kernel columns $\kappa(V, x_i)$, squared norms become kernel diagonals, iterates exist purely as convex weights (the exact cache refresh is reconstructed from the support's cached kernel columns), and the classifier is the kernel expansion of the witness-pair bisector. A ridge $K + I/C$ gives the kernel L_2 margin by the same diagonal trick. Validation: the linear kernel reproduces the Euclidean solver to 10^{-12}; RBF distances match the exact kernel QP to 10^{-9} and LIBSVM's hard-margin solution, via $\|w_{\mathcal{H}}\|^2 = (\alpha \circ y)^\top K (\alpha \circ y)$, $w_{\mathcal{H}}$ the feature-space normal, for LIBSVM's dual coefficients α and labels y, to 10^{-5}; kernel SMO matches the QP to 10^{-8}.

Appendix C. Experimental details

Solver settings and tolerance equivalence. ETA: block transfers with $k = 16$, guarded, away fallback, pairwise anti-zig-zag, stop at certified relative gap 10^{-3}, iteration cap 2×10^6, 600 s cap. Every result comes from the released code (schedule of Lemma 10, case-(b) fallback in every path). The tables and figures of real-data and synthetic timings were rerun in October, after this paper's acceptance, with three baseline-fairness changes: our SMO keeps every kernel row it computes (in the submission it had a 512-row cache), LIBLINEAR's dual solver is run beside its Newton solver, and the step ablation adds exact-search rows. Tables 3 and 8 and the September entries named in the hardware paragraph were not rerun. The submitted version's single-surviving-pair path took the possibly capacity-clipped MDM transfer directly, bypassing the fallback; the fix changes a block trajectory only on such iterations. Block assembly copies the k' cached columns into a $k' \times n$ array,

an $O(k'n)$ copy, before the $O(k'^2)$ products; screening (every 25 iterations) only in the LIN cell, where it never activated at 10^{-3} (Theorem 6 gives a sufficient condition only, which these runs stop before meeting). Our SMO: maximal-violating-pair selection, KKT gap 10^{-3}, same caps; hard margin as $C = 10^{12}$; kernel SMO on $K + I/C$; a row cache that keeps every row it computes, as ETA's column cache does (covtype in Table 2 still uses the 512-row cache). LIBSVM on the precomputed $K + I/C$ kernel (scikit-learn `SVC`, `kernel='precomputed'`, C=10^6, tolerance 10^{-3}, five seeds; `src/libsvm_kl2.py`) solves the same kernel L_2 margin: the RBF kernel with $1/C$ on the diagonal is built with BLAS in 1–3 s and the fit takes 0.5–2.2 s, with distances equal to K-ETA's to five digits. LIBLINEAR (scikit-learn 1.8 `LinearSVC` with its default `dual='auto'`, which selects the primal trust-region Newton solver on these datasets, not dual coordinate descent): squared hinge with parameter $C/2$, so that its objective equals $\frac{1}{2}\|w\|^2 + \frac{C}{2}\sum \xi_i^2$; tolerance 10^{-6}, chosen so that its final primal matches ETA's certified value on every cell where ETA converges except gisette at C=10 (a9a 687.07 vs 687.07; gisette 0.14900 vs 0.14893, inside ETA's certified 10^{-3}; ijcnn1 550.61 vs 550.61); this tolerance is why the Newton solver needs 17,550 iterations and 32 min on gisette at C=1, and 44,569 iterations and 1.2 h at C=10, where it nevertheless stops 4.2% above the certified optimum (0.15532 vs 0.14899). Table 4 sweeps the tolerance for both solvers: at the default 10^{-4} the Newton solver takes 3–40 s on gisette but stops 3–24% above the optimum, while the dual solver reaches it in 3–4.5 s at every tolerance; on a9a the order reverses. Intercept scaling 100, so LIBLINEAR regularises $b/100$ where the geometric formulation leaves b free (the primals agree to five digits); iteration cap 2×10^5. LIBSVM (scikit-learn `SVC`): tolerance 10^{-3}, 1000 MB cache, iteration cap 2×10^6 (reported as *maxiter* when hit); hard margins as $C = 10^6$; RBF kernel $\exp(-\|x - x'\|^2/d)$. Oracle counts: ETA counts Gram or kernel columns evaluated against a full class ($O(nd)$ or $O(n_k d)$ each); SMO counts kernel rows, the same unit. Columns are cached without eviction and the block Gram matrix Γ is assembled from four cached entries per pair, so peak cache memory is the column count times n_k doubles: 0.73 GB for the largest cell (covtype KL2, 9,100 columns), 0.61 GB for a9a KL2 and 0.36 GB for ijcnn1 KL2; measured peak resident memory in the isolated runs of Table 6 is 0.74 GB on gisette (data-dominated), 0.49–0.93 GB for ETA and 0.68–1.30 GB for BPCG on the kernel cells.

Data and seeds. a9a, w8a, ijcnn1 and gisette use the LIBSVM training and test files; covtype (binary, scaled) is split 75/25 by seed and its training part subsampled to 100,000 points. Features are standardised on the training set. Kernel cells use a class-balanced subsample of at most 5,000 points per class (w8a's minority class has 1,479 points, hence n_k = 6,479; gisette uses all 6,000). Seeds vary the covtype split, the kernel subsamples and the timing environment; on datasets with fixed splits the deterministic solvers return identical solutions across seeds, which is why several accuracy CIs are zero. Confidence intervals are t-based 95% intervals over seeds.

Hardware and concurrency. The October rerun used two dedicated cloud VMs with the same software (Linux, Python 3.14.7, NumPy 2.4.4 with OpenBLAS 0.3.31, SciPy 1.17.1, scikit-learn 1.8.0, `OMP_NUM_THREADS=1`, no other load). A 16-vCPU AMD EPYC 9754 (60 GB RAM) ran the regime battery (Table 1) and the synthetic benchmark (Figure 3) four single-threaded cells at a time, the L_2 battery (Table 2) up to four at a time, and the runs of Figure 2 one at a time. A 32-vCPU AMD EPYC 7B13 (128 GB RAM) ran the step ablation and the BPCG comparison (Tables 6 and 7) one configuration at a time, the LIBLINEAR sweep (Table 4) three processes at a time, the uncapped traces (Figure 1) one at a time, and covtype's LIBLINEAR entries. Comparisons are made within one machine except the trace-to-LIBLINEAR ratios of Section 5 and some entries of Table 5;

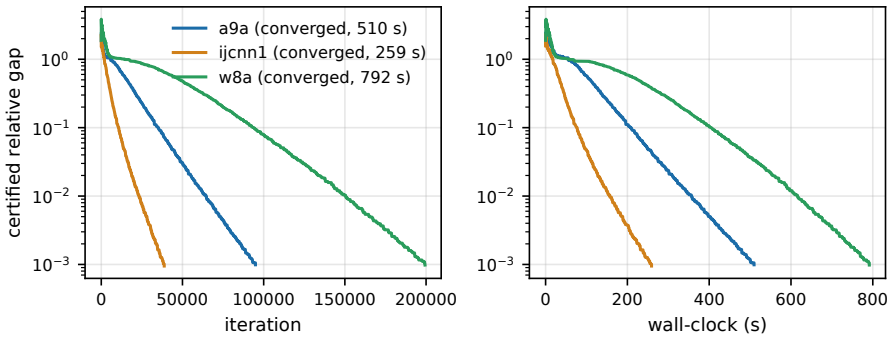


Figure 1: Certified relative gap of ETA on a9a, w8a and ijcnn1 (L_2, $C = 0.1$, uncapped, seed 0), versus iteration and wall-clock: linear-rate convergence in the dense-support regime, reaching the certified 10^{-3} in 259–792 s against the 600 s cap of Table 2 (AMD EPYC 7B13). Heuristically (δ has no known bound in terms of the class hulls), $n_s \approx 0.6n$ active atoms forming a near-regular simplex have pyramidal width $\sqrt{2/n_s}$ times the side [3], so $\rho \sim 1/n_s$ and the cost is $O(n^2)$-type.

cross-machine ratios can be off by more than $1.5\times$ (ETA on the gisette L_2 margin at C=1: 13.0 s in isolation on the EPYC 7B13, 20.6 s in the EPYC 9754 battery). September results, on a 32-vCPU Intel Xeon 2.6 GHz VM (122 GB RAM, Python 3.12): covtype's ETA and SMO entries in Table 2 (four cells at a time, SMO with a 512-row cache; a full cache on covtype's 100,000 training points needs about 80 GB, and the run was killed on both October machines), where both timed out on every cell; the LIBSVM-precomputed brackets of Table 1 and the gisette KL2 row of Table 7 (up to 19 concurrent processes); and Tables 3 and 8, whose absolute times carry an uncertain load, so only their within-table ratios should be read. Concurrent load can manufacture ratios: under September's 19-process load guarded k=1 appeared $1.3\times$ faster than MDM on gisette, a tie in isolation; only ratios whose intervals separate clearly should be read as results.

Reference solver in the KL2 cells. The hinge-loss `SVC(rbf, C=1)`, a different objective, took 4.5 ± 0.4 (a9a), 3.1 ± 0.3 (covtype), 75.1 ± 4.8 (gisette), 0.9 ± 0.1 (ijcnn1) and 2.2 ± 0.7 s (w8a), with test accuracies 0.789, 0.783, 0.977, 0.950 and 0.975, within one point of the L_2 solutions and on either side of them.

Step ablation, blocks and shrinking (synthetic). Iteration counts at $\varepsilon = 10^{-6}$ (single instances): d=30, n=200: toward 158,941, midpoint 148,996, away 106, pairwise 181; d=100, n=1,000: toward and midpoint fail within 300,000, away 15,886, pairwise 3,921. At the working tolerance 10^{-3} the pairwise/away gain over the midpoint heuristic is only $\sim 11\%$, which is why the heuristic sufficed in [2]. Block transfers cut iterations 3–25$\times$ relative to single MDM (e.g. 43,486 $\rightarrow$ 3,001 at k=16 on a dense-support L_2 instance at d=1,000) and are the only configuration that reaches the certified tolerance on MNIST odd-vs-even. Figure 2 shows the screening speed-ups and Figure 3 the consolidated comparison, five trials per class (mean $\pm$ 95% CI): on the four hard-margin classes the optimised solver ranges from $1.5\times$ faster than LIBSVM to $1.1\times$ slower (0.47 ± 0.06 vs 0.69 ± 0.15 s at d=1,000, the only separated intervals; 14.1 ± 1.5 vs 13.2 ± 1.9 s at d=10,000; 0.59 ± 0.11 vs

Table 2: L_2 soft-margin battery: five seeds, 600 s per solver for ETA and our SMO (our own implementation; LIBSVM does not solve this objective), LIBLINEAR (primal Newton solver, Appendix C) at tolerance 10^{-6} with no cap; released code with a full SMO row cache, up to four single-threaded cells at a time on the AMD EPYC 9754 machine, except covtype (below). Mean $\pm$ 95% CI, which on the fixed-split datasets (all but covtype) measures timing noise only (w8a, C=0.1: one LIBLINEAR seed took 71 s, the other four 12–14 s). "cap" = timed out in every seed; * four of five seeds converged within the cap, the mean counting the capped seed at 600 s. ETA's certified relative gap is reported at its stopping point (mean [min, max]); accuracies are those of the returned iterate. covtype: ETA and SMO from the September run on the earlier machine with SMO's 512-row cache, because a full cache needs about 80 GB (Appendix C); LIBLINEAR on an AMD EPYC 7B13, whose objective values match the September run's to every digit.

dataset ($n+m$ / d)	C	ETA (s)	SMO (s)	LIBLINEAR (s)	ETA gap at stop	acc. ETA / SMO / LIBLINEAR
gisette (6,000 / 5,000)	0.1	19.3 ± 2.4	16.7 ± 2.6	261 ± 4	9×10^{-4}	.981 / .981 / .981
	1	20.6 ± 3.5	15.9 ± 3.3	$1{,}893 \pm 41$	10×10^{-4}	.981 / .981 / .981
	10	20.9 ± 1.4	19.6 ± 1.1	$4{,}380 \pm 277$	10×10^{-4}	.981 / .981 / .978
a9a (32,561 / 123)	0.1	583 ± 17*	163 ± 4	1.9 ± 0.1	$(9.9\text{–}11) \times 10^{-4}$	.849 / .849 / .849
	1	cap	cap	3.2 ± 0.8	0.66 [0.55, 0.85]	.840 / .850 / .849
	10	cap	cap	1.1 ± 0.1	1.2 [1.2, 1.2]	.594 / .786 / .849
ijcnn1 (49,990 / 22)	0.1	336 ± 61	59.0 ± 7.4	0.1 ± 0.0	10×10^{-4}	.918 / .918 / .918
	1	cap	334 ± 11	0.2 ± 0.1	0.17 [0.16, 0.18]	.920 / .918 / .918
	10	cap	cap	0.2 ± 0.0	1.1 [0.99, 1.2]	.649 / .917 / .918
w8a (49,749 / 300)	0.1	cap	470 ± 50	24.4 ± 32.2	0.044 [0.016, 0.12]	.987 / .987 / .987
	1	cap	cap	6.8 ± 1.6	0.86 [0.84, 0.89]	.979 / .987 / .987
	10	cap	cap	8.5 ± 1.5	1.4 [1.4, 1.5]	.975 / .977 / .987
covtype (100,000 / 54)	0.1	cap	cap	12.8 ± 11.1	1.0 [1.0, 1.0]	.572 / .701 / .756
	1	cap	cap	3.2 ± 1.5	1.1 [1.0, 1.1]	.537 / .545 / .756
	10	cap	cap	2.5 ± 0.9	1.3 [1.3, 1.6]	.502 / .515 / .756

0.63 ± 0.15 s at ε=10^{-5}, where the paper-style configuration fails to converge in every trial; RBF 1.64 ± 0.42 vs 1.94 ± 0.34 s); on the synthetic L_2 margin (d=1,000, C=1) it ties LIBLINEAR (13.7 ± 11.0 vs 14.5 ± 5.0 s); on MNIST odd-vs-even it is 2.1× slower (135 ± 17 vs 63 ± 13 s; single MDM never converges) and on the ν-SVM 1.4× slower (41.8 ± 7.9 vs NuSVC's 30.9 ± 5.7 s). Distances agree within tolerance on every row.

The guarded single pair already halves the iterations of plain MDM on ijcnn1 (1.4× on gisette): the drop steps it takes (on 0.1–2.4% of its iterations) are what plain pairwise FW replaces by capacity-clipped swaps; an exact receiver search cuts iterations as much and, on the kernel cells, time by 1.8–1.9×. Blocks then cut iterations a further 3.8–7.0×. Away fallbacks that drop an index occur on 0.1–36% of guarded iterations, and their share grows with k: larger blocks add more indices per iteration, and the drops remove them. The time split says where fewer iterations buy time. On gisette, 82–89% of every configuration's time except away-step FW's is Gram-column computation ($O(nd)$ per new receiver, d=5,000), so all but away-step FW lie within 10% of one another at 12–13 s. On the kernel cells the columns are cheap and the rest is per-iteration $O(n)$ work: the score update and line search inside the step, the certificate in the remainder, and least of all the scan itself. That work falls with the iteration count, though not in proportion, because a block step updates the scores with up to k' columns; the net effect on ijcnn1 is 2.1× over MDM, 1.2× over exact-search

Table 3: Two-sided schedules on the synthetic instances of this appendix (n=5,000 per class, k=16): the unconditional V-then-W order of the submitted version versus the schedule of Lemma 10 (larger-gap side first, second side skipped after a drop). Drop steps and skipped second-side steps are counted over the run.

instance	schedule	time (s)	iterations	drops	skips	distance
hard margin, d=1000, ε=10^{-5}	unconditional	0.8	16	0	0	55.339365
hard margin, d=1000, ε=10^{-5}	Lemma 10	0.8	16	0	0	55.339365
L_2 overlap, d=100, C=1	unconditional	25.2	12,311	785	0	0.078400
L_2 overlap, d=100, C=1	Lemma 10	23.3	10,701	599	316	0.078400
L_2 overlap, d=1000, C=1	unconditional	12.9	2,901	182	0	0.578143
L_2 overlap, d=1000, C=1	Lemma 10	13.4	3,001	180	121	0.578143

Table 4: LIBLINEAR's two solvers across tolerances on gisette and a9a (`src/liblin_sweep.py`; five seeds, three processes at a time and nothing else running, AMD EPYC 7B13). Newton: `LinearSVC` with `dual='auto'`, which selects the primal trust-region Newton solver, the one the batteries ran; dual: `dual=True`, dual coordinate descent. Primal objective relative to the best known value P^* (0.14835 and 0.14893 on gisette at C=0.1 and 1, ETA's certified values; 687.07 and 6,869.7 on a9a); on these fixed splits every seed returns the same primal and accuracy. The dual solver hit its 2×10^5 iteration cap on a9a at C=1 and $\mathrm{tol} = 10^{-8}$.

dataset	C	tol	Newton (s)	Newton primal	dual (s)	dual primal	acc. Newton / dual
gisette	0.1	10^{-4}	40.1 ± 0.5	0.15345 (+3.4%)	3.0 ± 0.1	0.14835 (+0.0%)	.978 / .981
		3×10^{-5}	85.7 ± 0.8	0.14922 (+0.6%)	3.3 ± 0.2	0.14835 (+0.0%)	.980 / .981
		10^{-5}	151 ± 2	0.14842 (+0.05%)	3.7 ± 0.6	0.14835 (+0.0%)	.981 / .981
		10^{-6}	269 ± 2	0.14835 (+0.0%)	4.5 ± 0.7	0.14835 (+0.0%)	.981 / .981
gisette	1	10^{-4}	3.0 ± 0.2	0.18440 (+24%)	2.9 ± 0.2	0.14893 (+0.0%)	.974 / .981
		3×10^{-5}	4.0 ± 0.3	0.18343 (+23%)	3.2 ± 0.1	0.14893 (+0.0%)	.974 / .981
		10^{-5}	398 ± 18	0.15568 (+4.5%)	3.5 ± 0.1	0.14893 (+0.0%)	.977 / .981
		10^{-6}	$1{,}813 \pm 67$	0.14900 (+0.05%)	4.5 ± 0.7	0.14893 (+0.0%)	.981 / .981
a9a	0.1	10^{-4}	0.5 ± 0.1	687.08 (+0.001%)	131 ± 38	687.07 (+0.0%)	.849 / .849
		10^{-6}	1.8 ± 0.4	687.07 (+0.0%)	180 ± 44	687.07 (+0.0%)	.849 / .849
		10^{-8}	2.4 ± 0.5	687.07 (+0.0%)	226 ± 50	687.07 (+0.0%)	.849 / .849
a9a	1	10^{-4}	0.6 ± 0.1	6,869.7 (+0.001%)	$1{,}291 \pm 416$	6,869.7 (+0.0%)	.850 / .849
		10^{-6}	2.5 ± 0.3	6,869.7 (+0.0%)	$1{,}828 \pm 271$	6,869.7 (+0.0%)	.849 / .849
		10^{-8}	5.9 ± 0.8	6,869.7 (+0.0%)	$1{,}852 \pm 249$	6,869.7 (+0.0%)	.849 / .849

MDM and 4.4× over BPCG. Away-step FW with exact search needs 30–110× more iterations than plain MDM where it converges and times out on three kernel cells.

How much blocks help, and choosing k. For uncapped pairs $S = \sum_j b_j^2$, so $\hat{\eta} = (D/S - 1)/(k' - 1)$ is the effective pair correlation of Proposition 4, and the guard computes it at no cost. Instrumented seed-0 runs on the five real cells of Table 6 (`src/block_diagnostics.py`, September, one process at a time, $k \in \{1, 4, 16, 32\}$) give median $\hat{\eta} = 0.07$ (gisette), 0.11 (a9a), 0.17 (w8a), 0.26 (ijcnn1) and 0.29 (covtype). The iteration cut of k=16 over plain MDM is larger at low correlation (15× on a9a, 19× on w8a) than at high (9× on ijcnn1, 8× on covtype), and

Table 5: Time to 1% of the optimum, seed-0 traces. The ETA columns measure *dual* progress and LIBLINEAR's column *primal* progress, so this is not a matched primal comparison (the primal of ETA's returned (w, b) is not recorded in the traces). For ETA the certified dual value $2/\mathrm{UB}_t^2$ increases to the optimum, so $s_t = 1 - (2/\mathrm{UB}_t^2)/P^*$ with P^* the best known primal (the lower of LIBLINEAR's best value and ETA's certified value); "certified 1%" is the first time the certificate itself proves $1 - \mathrm{LB}_t^2/\mathrm{UB}_t^2 \le 1\%$. ETA's a9a, w8a and ijcnn1 times are from the uncapped runs of Figure 1 (`src/time_to_tol.py`); its gisette times are from the battery traces of Table 2. Machines: ETA's gisette entries and LIBLINEAR's upper bounds come from the EPYC 9754, ETA's a9a, w8a and ijcnn1 entries and LIBLINEAR's swept times from the EPYC 7B13. LIBLINEAR: the mean time of the coarsest tolerance of Table 4 whose primal is within 1% of P^* in every seed, for the Newton and the dual solver; where the sweep did not run, the matched 10^{-6} Newton run of Table 2, an upper bound (on gisette at C=10 that run stops 4.2% above P^*). On the remaining cells ETA is not within 1% at the 600 s cap (s = 3–89%).

dataset	C	P^*	ETA 1%	ETA 0.1%	cert. 1%	cert. 10^{-3}	LIBLINEAR 1%, Newton / dual
gisette	0.1	0.14835	17 s	18 s	19 s	19 s	86 s / 3.0 s
gisette	1	0.14893	20 s	21 s	21 s	22 s	1,813 s / 2.9 s
gisette	10	0.14899	20 s	21 s	22 s	22 s	none at 10^{-6} / not run
a9a	0.1	687.07	218 s	295 s	402 s	510 s	0.5 s / 131 s
w8a	0.1	108.80	435 s	559 s	670 s	792 s	$\le$ 24 s / not run
ijcnn1	0.1	550.61	80 s	125 s	189 s	259 s	$\le$ 0.1 s / not run

only $6\times$ on gisette despite its low correlation, because drop steps take 36% of its block iterations. Proposition 4 guarantees a 16-pair block of equal pair gains 3–8 transfers' worth at the measured correlations, against 16 for orthogonal pairs. On the synthetic instances the cut grows with d ($6.8\times$ at d=100, $14.5\times$ at d=1,000, Table 8), as expected if random difference directions become more orthogonal in higher dimension; we did not measure $\hat{\eta}$ there. If an iteration costs $a + bk$ (scan plus per-pair work) and a block is worth $k/(1 + \eta(k-1))$ transfers, the cost per transfer is minimised at $k^* = \sqrt{(a/b)(1-\eta)/\eta}$; with a and b fitted per cell and $\hat{\eta}$ for η this gives $k^* = 7$–14, to be capped by the support size and the column cache. In the September timings the best was k=4 on gisette and ijcnn1 and k=16 on the other three cells, within $1.3\times$ of each other; on gisette, where columns take 93% of the time, no k helps.

BPCG here is the blended pairwise step of [12] on the atoms $v_i - w_j$ of Z with the separable LMO of Lemma 7, exact line searches and the certificate of Lemma 5; its scores are maintained from the same cached columns, so column counts are comparable. It needs 11–$35\times$ more iterations than block ETA on every cell, each an $O(n + m)$ scan; the block's fewer scans buy 4.4–$7.1\times$ in wall-clock on the four isolated kernel cells ($2.5\times$ on gisette's, in the September run), where scans dominate, and nothing where columns do (gisette L_2). BPCG runs on the product atoms $v_i - w_j$, so the ratio compares parametrisation as well as step rule.

Appendix D. Extended related work

Aggregated directions. Boosted FW [13] builds its direction by greedily aligning a combination of atoms with the negative gradient, at several oracle calls per iteration, and inherits the FW rate

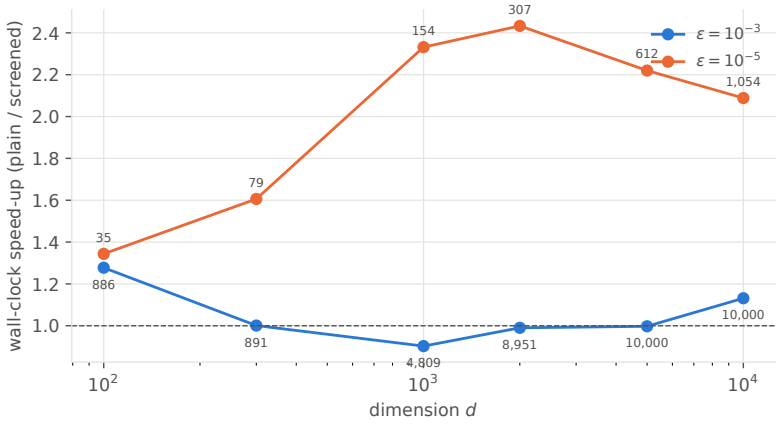


Figure 2: Safe screening: wall-clock speed-up vs dimension at two tolerances on the separated synthetic protocol; labels are surviving points of 10,000. Made with the toward-step solver of the development phase (`src/shrink_benchmark.py`); the rule and its activation do not depend on the step.

through an alignment condition; our block fixes the pair coefficients γ_j from one scan and relies on the guard, not on alignment, for its rate.

Swap steps. Blended pairwise conditional gradients [12] also remove pairwise FW's swap-step bound with a guard, there between the pairwise step and a FW step, and [14] avoid bad steps with a short-step chain. Our guard compares an aggregated block of k transfers with the single MDM step and falls back to an away step, so the drop-step count of away-step FW applies unchanged. Block-coordinate FW [9] blocks over *coordinates* of a product domain rather than aggregating pairs on one polytope.

Screening and identification. Gap-safe rules [15, 17, 18] screen with a radius from the duality gap and identify the support in finite time; the active-set complexity of away-step FW [19] and the strict-complementarity bounds of [20] give the same identification from the algorithm's side. Theorem 6 is the polytope-distance instance, with constants tied to the TA certificate; the SVM rules of [10, 11] and the exact data reduction of [16] are its ancestors.

Relation to other FW variants. The toward, away, pairwise and block rows of the step ablation in Appendix C are Frank–Wolfe, away-step FW, pairwise FW and our block step on the same polytope and objective; blended pairwise conditional gradients (BPCG) [12] is compared on real data in Table 7; Wolfe's minimum-norm-point method [26] was not run. BPCG's blending rule compares the away and FW gaps and its linear rate has the same pyramidal-width form as Theorem 3; our case (b) instead drops when the away step is clipped and otherwise compares the realised gains of the block, MDM, away and toward candidates, so each non-drop step is at least as good as the best single candidate, and it is the away fallback rather than the blending that yields the drop-step count. The facial distance of [27] is a computable alternative to the pyramidal width; a bound on δ for the difference atom set $A = \{v_i - w_j\}$ in terms of the two class hulls is open.

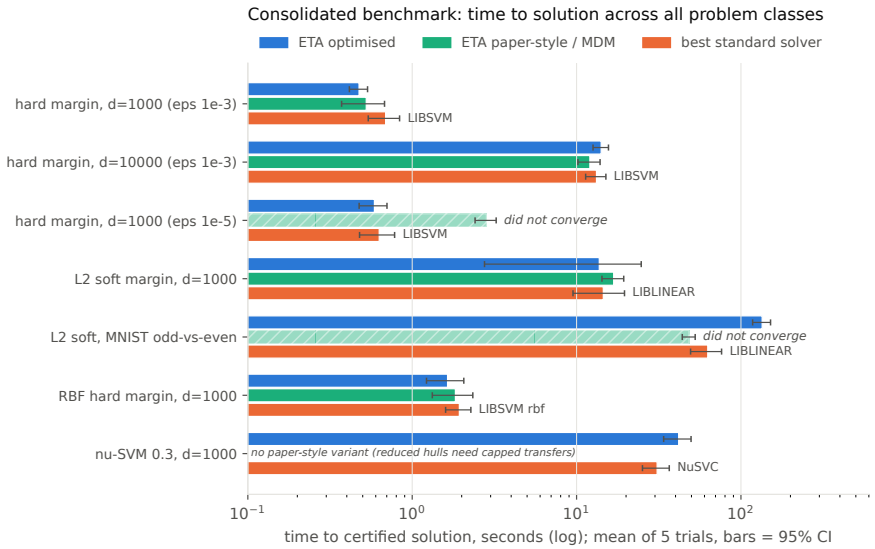


Figure 3: Time to certified solution on seven problem classes: optimised ETA vs the paper-style configuration vs the best standard solver per class; five trials per class (October rerun, AMD EPYC 9754), mean with 95% CI. The ν-SVM row has no paper-style bar because the toward-step algorithm of [2] has no reduced-hull variant (reduced hulls need capped transfers).

Synthetic results in brief. Away and pairwise steps converge at $\varepsilon = 10^{-6}$ where the toward and midpoint steps of [2] do not. Screening reduces $10{,}000$ points to 35–$1{,}054$ survivors at $\varepsilon = 10^{-5}$. On seven synthetic and small-real problem classes (Figure 3, five trials each) the optimised solver is between $1.5\times$ faster and $1.1\times$ slower than LIBSVM on the four hard-margin classes, ties LIBLINEAR on the synthetic L_2 margin, and is 1.4–$2.1\times$ slower than NuSVC and LIBLINEAR on the ν-SVM and MNIST classes. On the four classes where single MDM also converges, blocks and guard change wall-clock by 0.9–$1.2\times$; their clear wins are the two classes (tight tolerance, MNIST) where single MDM does not converge at all.

Appendix E. Numerical verification of the analysis

Lemma 1 was checked on $4{,}000$ random ensembles including heavily clipped and boundary cases (worst slack at machine precision); Proposition 4 on 500 near-orthogonal ensembles; Lemma 5 to 10^{-9} at arbitrary iterates; Lemma 10 on instrumented two-sided runs (no drop iteration added a support index, and every non-drop iteration met the gain bound with M replaced by the computable upper bound $\operatorname{diam} V + \operatorname{diam} W$); and Theorem 6 on instances whose optimal face was computed by an exact QP: at every activating round all zero-weight non-face points were screened, and the final working set equalled the optimal face exactly. Over $5{,}000$ instrumented live block steps none increased the objective. All checks are test files in the released code.

Table 6: Step ablation on real data with a time split, isolated runs (`src/step_profile.py`: three seeds, each configuration alone in its own process on an otherwise idle AMD EPYC 7B13, with the final code): the gisette L_2 margin (C=1, d=5,000) and the four kernel L_2 cells of Table 1. MDM is plain pairwise FW (single transfers, no guard, no fallback) with the prioritised receiver search and schedule of the other rows; "exact" rows replace that search by an exact linear minimisation over all points; AFW is away-step FW with exact search; guarded k=1 is Algorithm 1 with a single pair (guard and case-(b) fallback active); BPCG as in Table 7. Split of the ETA time: scan = the score scan and pair selection; col = Gram or kernel column computation; step = the step itself (block assembly, guard, line search, cache and score update); rest = certificate and bounds. Drops = away fallbacks that removed a support index. Every run converged to the certified gap with identical distance and accuracy per dataset and seed, except AFW on the ijcnn1, a9a and covtype kernel cells, which timed out at 600 s with certified gaps of 0.0012–0.004 (iterations and columns at the cap shown). Iteration, column and drop counts equal those of the September runs for every configuration both contain except plain MDM, whose prioritised search varies between runs (up to 5% across seeds, under 0.3% between the two runs' means); only the times changed. Peak resident memory: 0.74 GB on gisette, 0.49–0.93 GB for ETA and 0.68–1.30 GB for BPCG on the kernel cells. Columns take 82–89% of every gisette configuration's time except AFW's, so those rows should be read as a tie.

instance	step	k	time (s)	scan	col	step	rest	iterations	columns	drops
gisette L_2	MDM	1	12.1 ± 0.2	0.2	10.2	0.8	0.9	4,946	1,300	0
gisette L_2	MDM, exact	1	12.0 ± 0.2	0.1	10.6	0.7	0.7	3,546	1,341	0
gisette L_2	guarded	1	12.8 ± 0.1	0.2	10.5	1.5	0.6	3,471	1,334	82
gisette L_2	guarded, exact	1	12.9 ± 0.4	0.1	10.7	1.5	0.7	3,416	1,341	87
gisette L_2	AFW, exact	1	206.7 ± 5.3	7.6	10.8	90.9	97.3	527,108	1,340	0
gisette L_2	block	16	13.0 ± 0.1	0.1	11.6	1.2	0.2	831	1,480	301
gisette L_2	BPCG	—	13.2 ± 0.1					10,470	1,322	87
ijcnn1 kernel L_2	MDM	1	7.8 ± 0.5	1.2	0.8	2.2	3.6	12,763	4,523	0
ijcnn1 kernel L_2	MDM, exact	1	4.4 ± 0.2	0.2	0.8	1.2	2.3	5,528	4,519	0
ijcnn1 kernel L_2	guarded	1	8.1 ± 0.4	0.8	0.7	4.1	2.6	6,118	4,514	14
ijcnn1 kernel L_2	guarded, exact	1	7.4 ± 0.5	0.1	0.7	3.9	2.6	6,070	4,520	14
ijcnn1 kernel L_2	AFW, exact	1	> 600	11.8	0.7	254.1	333.5	658,667	4,504	0
ijcnn1 kernel L_2	block	16	3.8 ± 0.3	0.2	0.6	2.3	0.7	1,396	4,590	98
ijcnn1 kernel L_2	BPCG	—	16.6 ± 1.5					18,034	4,512	19
w8a kernel L_2	MDM	1	6.6 ± 0.7	0.9	1.6	1.8	2.3	11,491	3,588	0
w8a kernel L_2	MDM, exact	1	3.4 ± 0.2	0.1	1.4	0.8	1.1	4,307	3,585	0
w8a kernel L_2	guarded	1	5.2 ± 0.5	0.4	1.4	2.4	1.1	4,243	3,581	8
w8a kernel L_2	guarded, exact	1	5.0 ± 0.4	0.1	1.4	2.3	1.1	4,248	3,586	11
w8a kernel L_2	AFW, exact	1	225.7 ± 3.4	5.3	1.5	107.1	111.7	350,050	3,569	0
w8a kernel L_2	block	16	2.5 ± 0.2	0.1	1.3	0.9	0.2	603	3,591	50
w8a kernel L_2	BPCG	—	13.4 ± 1.2					12,342	3,577	12
a9a kernel L_2	MDM	1	23.3 ± 0.3	3.1	3.1	5.8	11.3	22,879	7,609	0
a9a kernel L_2	MDM, exact	1	12.8 ± 0.1	0.3	2.8	3.1	6.6	9,616	7,608	0
a9a kernel L_2	guarded	1	19.5 ± 1.1	1.7	2.7	8.5	6.7	9,879	7,611	27
a9a kernel L_2	guarded, exact	1	18.0 ± 0.4	0.2	2.6	8.3	6.8	9,852	7,607	22
a9a kernel L_2	AFW, exact	1	> 600	7.4	3.0	245.6	344.2	412,530	7,543	0
a9a kernel L_2	block	16	6.9 ± 0.6	0.3	2.4	3.0	1.2	1,489	7,616	35
a9a kernel L_2	BPCG	—	49.1 ± 1.5					28,653	7,605	24
covtype kernel L_2	MDM	1	29.3 ± 1.7	3.9	2.2	7.5	15.7	26,748	9,067	0
covtype kernel L_2	MDM, exact	1	16.2 ± 0.4	0.3	2.0	4.3	9.6	11,789	9,066	0
covtype kernel L_2	guarded	1	26.2 ± 1.8	2.3	1.8	11.9	10.2	12,544	9,069	12
covtype kernel L_2	guarded, exact	1	24.6 ± 1.3	0.3	1.9	11.9	10.6	12,602	9,068	14
covtype kernel L_2	AFW, exact	1	> 600	6.4	2.0	243.8	348.0	354,002	9,026	0
covtype kernel L_2	block	16	12.2 ± 0.2	0.7	1.6	6.7	3.1	3,336	9,074	26
covtype kernel L_2	BPCG	—	70.0 ± 4.6					38,422	9,065	15

Table 7: BPCG [12] versus block ETA (k=16) on the same objective with the same certificate (ε=10^{-3}), the same column cache and the same 600 s cap (solver `src/bpcg.py`): the isolated runs of Table 6 (three seeds, AMD EPYC 7B13), and for gisette KL2, which those runs do not include, the September run (`src/bpcg_baseline.py`, three seeds, concurrent with the regime battery on the earlier machine), marked †. Every cell converged; distances agree per cell to five digits and test accuracies to 10^{-4}. Columns are kernel or Gram columns computed; drops count BPCG's capped local steps and ETA's away drops.

cell	solver	time (s)	iterations	columns	drops
gisette L_2, C=1	BPCG	13.2 ± 0.1	10,470	1,322	87
gisette L_2, C=1	ETA	13.0 ± 0.1	831	1,480	301
gisette KL2†	BPCG	213 ± 28	8,702	3,641	3
gisette KL2†	ETA	83.9 ± 19.0	251	3,653	4
w8a KL2	BPCG	13.4 ± 1.2	12,342	3,577	12
w8a KL2	ETA	2.5 ± 0.2	603	3,591	50
a9a KL2	BPCG	49.1 ± 1.5	28,653	7,605	24
a9a KL2	ETA	6.9 ± 0.6	1,489	7,616	35
covtype KL2	BPCG	70.0 ± 4.6	38,422	9,065	15
covtype KL2	ETA	12.2 ± 0.2	3,336	9,074	26
ijcnn1 KL2	BPCG	16.6 ± 1.5	18,034	4,512	19
ijcnn1 KL2	ETA	3.8 ± 0.3	1,396	4,590	98

Table 8: Block-size ablation on the synthetic instances (released code, September run): single MDM (k=1) versus blocks of k transfers. Iterations fall with k; wall-clock does not fall in proportion, because each block iteration touches more columns.

instance	k	time (s)	iterations	columns	drops
hard margin, d=1000, ε=10^{-5}	1	0.6	301	104	0
hard margin, d=1000, ε=10^{-5}	4	0.6	36	103	0
hard margin, d=1000, ε=10^{-5}	16	0.5	16	103	0
hard margin, d=1000, ε=10^{-5}	32	0.6	16	109	0
L_2 overlap, d=100, C=1	1	26.3	72,546	1,345	0
L_2 overlap, d=100, C=1	4	27.2	24,606	1,632	416
L_2 overlap, d=100, C=1	16	23.9	10,701	2,404	599
L_2 overlap, d=100, C=1	32	27.4	7,606	3,076	305
L_2 overlap, d=1000, C=1	1	24.8	43,486	1,242	0
L_2 overlap, d=1000, C=1	4	17.7	10,071	1,330	177
L_2 overlap, d=1000, C=1	16	13.5	3,001	1,348	180
L_2 overlap, d=1000, C=1	32	13.1	1,741	1,396	170